

HW 6

Theo McArn

Question 1:

On the Mountain Car task, how do Monte Carlo methods perform relative to Temporal Difference methods?

Monte Carlo methods rely on the agent fully completing an episode in order for there to be any policy improvement. However, for environments like the Mountain Car task, where the episode only terminates once the car reaches the top of the mountain, and because it requires a clear strategy to reach this terminal point, rather than random actions, it is very rare that the car will reach the top with the initial random policy. As a result, the agent is never able to make significant updates to the policy to improve.

Conversely, with TD-Learning, the Q-function gets updated on every step, bootstrapping based on the adjacent states. As a result, termination is not needed to do policy improvement and therefore it can accelerate its learning as it improves.

Question 2:

For Semi-gradient Expected SARSA:

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

$S, A \leftarrow$ initial state and action of episode (e.g., ε -greedy)

Loop for each step of episode:

Take action A , observe R, S'

If S' is terminal:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$$

Go to next episode

Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε -greedy)

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [r + \gamma \mathbb{E}_{\pi} [\hat{q}(S', A', \mathbf{w})] - \hat{q}(S, A, \mathbf{w})] \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})$$

$f(S, A)$

$S \leftarrow S'$

$A \leftarrow A'$

For Semi-gradient Q-Learning:

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Loop for each step of episode:

Choose A from S using policy derived from Q (e.g., ε -greedy)

Take action A , observe R, S'

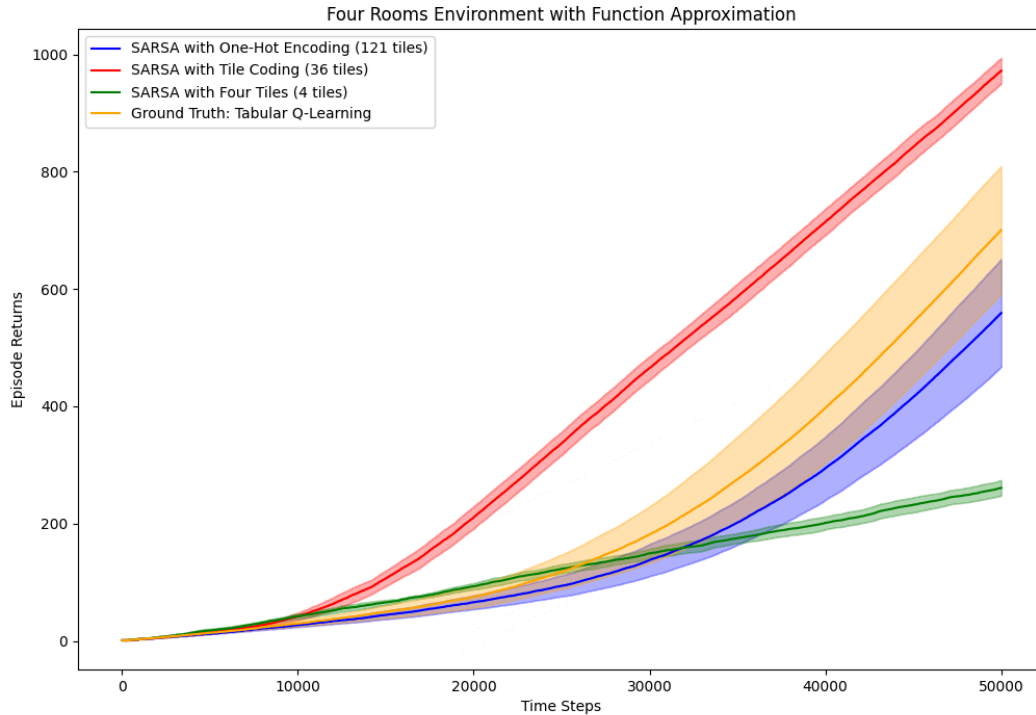
$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [r + \gamma \max_a [\hat{q}(S', a, \mathbf{w})] - \hat{q}(S, A, \mathbf{w})] \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})$$

$f(S, A)$

$S \leftarrow S'$

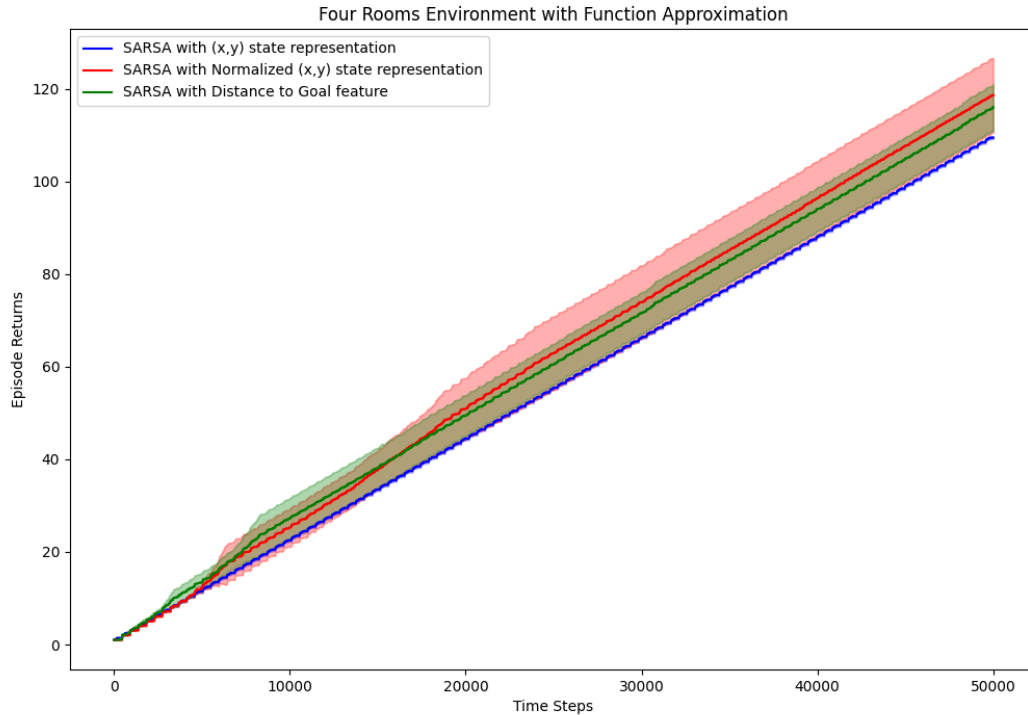
until S is terminal

Question 3:



Is aggregating the states helping to boost the learning speed? Shall we always aggregate the states as much as possible to learn better policy?

From this experiment, it seems that aggregating the states does boost learning speeds to a certain amount, as it allows for a more generalized state space, which can be used to make more generalizable actions. This can be seen when states were aggregated in (2×2) windows, in red. However, if there is too much aggregation, this results in a loss of resolution and information that results in poor learning, since there are not enough features to fit the highly non-linear Q-function. This result can be seen when trying to aggregate all 121 states into only 4, in green.



Why is the constant feature necessary? What do you think happens without it? Also describe how you incorporated actions into your features.

The constant feature is the bias term in the function approximation. Without this bias term, the approximated function would have to pass through the origin, meaning when all the features are zero, the q-value would always be zero, independent of the weights. For the two SARSA examples with only x and y features, this would mean that the q-value for any action from the starting state would be zero, no matter what the two weights are. By adding the bias term, this constraint is lifted and the approximated function can better fit the true Q-function.

In order for this function approximation to account for the different actions selected, I expanded each feature vector to a larger vector such that each action has its own dedicated segment of base features. For example, for the $(x, y, 1)$ representation, the full feature vector has 12 entries ($3 \text{ base features} \times 4 \text{ actions}$). When computing features for a given state-action pair, only the 3-element block

corresponding to that action is populated with the state features; all other entries are zero.

For example, given state (x, y) and the action index 0, the feature vector is:

```
f(s, a0) = [x, y, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

And for action index 1:

```
f(s, a1) = [0, 0, 0, x, y, 1, 0, 0, 0, 0, 0, 0]
```

Since $Q(s, a) = w^T * f(s, a)$, this structure means each action effectively has its own independent linear function of the state. For instance, $Q(s, a0) = w0*x + w1*y + w2$ while $Q(s, a1) = w3*x + w4*y + w5$, with no parameter sharing between actions. This allows the agent to learn different action preferences based on position, which would be impossible if the action were encoded within the single feature vector.

How do your results with the above features compare with state aggregation? If there is a significant performance difference, try to come up with explanations for it.

These features performed far worse than the initial state aggregation features. At first, I thought that this was surprising, but this reduced performance makes sense. With this smaller feature space, there are only 12 weights, plus the bias term which are being used to map a highly non-linear Q-function. This is not enough parameters to fully approximate this function and as a result, learning fails. Additionally, by using the raw, unnormalized position values as features, these large features create a large gradient which results in drastic changes to the weights and causes overshooting which in some cases resulted in the Q-value reaching the capacity of numerical instability and this caused errors. As a result, I had to decrease the step size dramatically, which further reduced learning ability.

Comments on Normalized Position and Distance Features

Similar to using the raw x and y positions as features, I also assumed that these features would result in even better learning. However, it is clear as before, that the parameterization is too small and as a result there is clear underfitting. The normalization did however improve the gradient issues which I mentioned before, which can be seen in the plot as it slightly outperforms the unnormalized version. I was particularly surprised by the results from learning with the distance feature. I

assumed having the extra feature would be beneficial, additionally because this sort of heuristic is useful in more traditional deterministic path planning algorithms like A*. However, it seems that learning using this additional feature performed worse than omitting it. My hypothesis is that the distance function creates mixed signals that gives the agent conflicting behaviors. Consider a state right next to the goal but on the other side of a wall. The distance function will return a low value indicating that the agent is very close to the goal, but the actual path might require going all the way around through a doorway. This feature cannot distinguish between this false positive, and when the agent truly is close to the goal. The agent is trying to fit a single constant weight to this feature, but the meaning of this feature isn't consistent. Sometimes small distance means a very high q-value, and sometimes it doesn't. As a result the optimization step keeps updating this corresponding weight in conflicting directions, resulting in an unconverged approximated function.